

Cora

Acompañamiento de salud para mujeres, personalizado por etapa de vida — pensado para Nicaragua.

Expo 57

React Native 0.86

Supabase · Postgres + RLS

Licencia MIT

Este documento corresponde al ítem "**Desarrollo — README técnico**" de la rúbrica del hackathon: descripción general, tecnologías utilizadas, instalación básica y ejecución del sistema.

Repositorio completo: github.com/eduardoevz/Volcanic-2026

1. Descripción general

La mayoría de apps de salud femenina son calendarios menstruales genéricos, en inglés, sin adaptación al contexto local. **Cora** acompaña a una mujer nicaragüense a lo largo de todas sus etapas de vida — adolescencia, adultez, embarazo, perimenopausia — adaptando contenido, herramientas y tono a la etapa que ella misma declara. La personalización por etapa de vida es el eje del producto: el calendario, la biblioteca educativa, la mascota y el asistente de IA existen para sostener esa idea, no al revés.

Cora **no diagnostica, no reemplaza atención médica y no comparte datos con terceros** — toda superficie con contenido clínico muestra un descargo visible de "esto no es un diagnóstico".

Funcionalidades principales

Seguimiento de salud

- Registro diario de síntomas, ánimo y energía, con calendario de ciclo menstrual
- Predicción de ciclo y ventana fértil, detección de irregularidades y señales de alerta (sin nombrar nunca una condición médica — solo deriva a hablar con un profesional)
- Seguimiento de embarazo (semana, trimestre, fecha probable de parto) y agenda de citas médicas
- Estadísticas y tendencias del ciclo
- Resumen médico exportable a PDF, compartible con un profesional de salud

IA y biblioteca educativa

- Asistente de IA (Gemini) con guardrails en capas: nunca diagnostica ni prescribe, cita fuentes reales de la biblioteca, y deriva ante señales de alerta en vez de responder
- Biblioteca con 38 artículos publicados, revisados y citados, filtrados por etapa de vida, con búsqueda semántica y audio educativo

Acompañamiento y personalización

- Círculo de acompañamiento familiar con permisos granulares por usuaria (RLS: sin invitación activa, cero acceso — nunca implícito)
- Directorio de centros de salud y especialistas (con consentimiento explícito para publicar)
- Mascota "pitahaya" que evoluciona con hábitos de autocuidado
- Recordatorios y notificaciones push reales
- Modo oscuro, y arquitectura de idioma lista para español/miskito/mayangna — hoy solo el contenido en español está completo; miskito y mayangna caen a español por falta de traducciones reales, documentado así en vez de simulado

Cuenta y seguridad

- Auth por correo/contraseña y Google OAuth, recuperación de contraseña, sesión y caché cifradas localmente
- Row Level Security en las 26 tablas de la base de datos, verificado directamente contra el proyecto (pg_tables/pg_policies) — auditoría detallada tabla por tabla en [docs/RLS_AUDIT.md](#)

2. Tecnologías utilizadas

CAPA	TECNOLOGÍA
App	Expo 57 + React Native 0.86 + React 19, TypeScript, expo-router
Estado y datos	Zustand, TanStack Query (caché offline persistida en AsyncStorage)
Formularios	react-hook-form + Zod
i18n	i18next / react-i18next
Backend	Supabase (Postgres, Row Level Security, Auth, Storage) — 26 migraciones versionadas, 26 tablas
Edge Functions	cora-ai (asistente), embed-content / search-articles-semantic (búsqueda semántica con pgvector), send-push (notificaciones)
IA	Gemini, vía Edge Function propia con guardrails determinísticos + de prompt
Notificaciones	expo-notifications + Expo Push API / FCM
Otros	expo-print (PDF), expo-audio (audio educativo), expo-secure-store + cifrado AES local
Calidad	Jest + React Native Testing Library, ESLint, TypeScript estricto, pgTAP (RLS), Maestro (E2E), GitHub Actions

Estructura del repositorio

```
Volcanic-2026/
├─ cora/                # App Expo/React Native
│  └─ app/              # Rutas (expo-router)
│     └─ src/features/  # Un módulo por feature (auth, tracking, assistant, family, ...)
│        └─ src/ui/     # Componentes y tema (claro/oscuro)
│           └─ e2e/      # Flujos Maestro
│              └─ supabase/ # Migraciones, Edge Functions y pruebas pgTAP
├─ docs/                # Documentación real del proceso
├─ entregables/         # Documentos exportados (README, estudio de BD, etc.)
├─ Entregables_Hackathon/ # Entregables oficiales según la rúbrica del hackathon
└─ releases/            # APKs de release listos para instalar
```

3. Instalación básica y ejecución del sistema

La app vive en `cora/` (Expo + React Native). Las carpetas nativas `android/` e `ios/` no están versionadas — se generan localmente con `prebuild`.

1. Clonar e instalar dependencias

```
git clone https://github.com/eduardoevz/Volcanic-2026.git
cd Volcanic-2026/cora
npm install
```

2. Configurar variables de entorno

```
cp .env.example .env.local
```

Completar `EXPO_PUBLIC_SUPABASE_URL` y `EXPO_PUBLIC_SUPABASE_ANON_KEY` con las credenciales del proyecto de Supabase.

3. Generar el proyecto nativo de Android

```
npx expo prebuild
```

4. Ejecutar

```
npx expo run:android # build + instala + abre Metro
# o, con el dev client ya instalado:
npm run start
```

5. Probar sin compilar

Hay un APK de release firmado en `releases/`, instalable directamente en un dispositivo Android (sideload) — no requiere Metro, cable USB, ni computadora corriendo. También publicado como GitHub Release para descarga directa.

6. Otros comandos útiles (dentro de cora/)

```
npm run lint      # ESLint
npm run typecheck # tsc --noEmit
npm test          # Jest
npm run test:coverage
```

Si `EXPO_PUBLIC_AI MOCK=true` está activo en `.env.local`, Cora IA responde con respuestas pregrabadas en vez de llamar a la Edge Function — útil si el proveedor de IA falla o va lento.

Calidad y pruebas

22 suites Jest / 142 tests (unitarias, de componentes e integración), guardrails de IA verificados con Gemini mockeado, suites pgTAP para Row Level Security, flujos E2E con Maestro, e integración continua en GitHub Actions (lint + typecheck + cobertura en cada PR). Detalle completo en `docs/TESTING.md`.

Documentación del proceso

Este proyecto documenta su propio proceso de construcción, fase por fase, con hallazgos reales (bugs encontrados, decisiones tomadas, limitaciones honestas) en vez de solo el resultado final:

- `docs/PROGRESO.md` — registro completo, fase por fase
- `docs/PLAN_DE_IMPLEMENTACION.md` — visión, alcance y roadmap
- `docs/RLS_AUDIT.md` — auditoría de seguridad de la base de datos
- `docs/AI_GUARDRAILS.md` — batería de verificación del asistente de IA
- `docs/CONVENCIONES.md` — convenciones de código y de marca
- `docs/DEMO_SCRIPT.md` — guion de demo del hackathon

Licencia

MIT